Curs 12

19.12.2007

Sumar

- ✓ Pointeri (şi tablouri, şiruri, structuri, funcţii)
  - Alocarea memoriei
  - Liste simplu înlănţuite

11. Pointeri

Pointeri

- Un *pointer* este o **variabilă** care conţine **adresa** unei alte variabile.
- Sintaxa de declarare a pointerilor este următoarea:

  tipVarPtr *nume_var;

  nume_var este un *pointer* la o valoare de tip ***tipVarPtr***

- *Pointerii* sunt utili în limbajul C pentru **alocarea dinamică** a memoriei (pentru crearea structurilor de date dinamice: liste, arbori, …).
- *"C este consistent şi constant cu aritmetica pointerilor; pointerii, tablourile şi aritmetica adresării constituie punctul forte al limbajului"* (K&R).

Pointeri

- Un ***pointer*** este o variabilă care conţine adresa unei alte variabile.

```
int var1;
…
var1=33;…
printf("%d",var1);
```

- Variabila 'var1' poate fi referită prin valoarea sa sau indicând către adresa sa, &var1.
- & - indică adresa
- * - indică ce se află la o anumită adresă

int *pointer;	declar un pointer către o variabilă de tip întreg
pointer=&var1;	pointer1 este o variabilă care stochează adresa variabilei var1
var2=*pointer;	⇔var2=var1;
printf("%d", *pointer);	

ptr1.c

Pointeri - exemple

- int *PtrInt; - declararea unei variabile *pointer* către un întreg
  Idem: int* PtrInt; sau int * PtrInt;
- char *PtrChar; - declară un *pointer* către un caracter
- struct ex_struct *PtrStr1,*PtrStr2 - declară doi *pointeri* către structura 'ex_struct'
- struct exemplu **PtrPtrEx; - *pointer* la un *pointer* la o structură 'exemplu';
- struct exemplu tabel[100]; - un tabel de 100 de structuri 'exemplu'
- struct exemplu *tabelPtr[100]; - un tabel de 100 de *pointeri* la structura 'exemplu'

Operatori

- ✓ ***** dereferenţiere (dereferencing) – aplicat unui argument are ca efect extragerea conţinutului zonei de memorie indicate de pointer;
- ✓ **->** dereferenţiere şi selecţia unei componente (câmp al) a unei structuri
- ✓ **&** indică adresa unei variabile (creează o referinţă).

Pointeri – utilitate

- ✓ pentru lucrul cu tablouri, şiruri de caractere, funcţii (care trebuie să transmită mai mult de o valoare în funcţia apelantă);
- ✓ pentru optimizarea programelor: rapid, mai puţină memorie;
- ✓ permit lucrul direct cu anumite locaţii de memorie (în aplicaţii cu microcontrolere).

Exemple

- ✓ Declararea unui *pointer* are ca efect alocarea de spaţiu în memorie pentru *pointer*; NU se alocă spaţiu pentru entitatea către care indică *pointer*ul.

```
int *p;

*p = 20;
```

➢ ‘p’ nu indică încă spre un întreg, ci undeva spre o entitate din memorie; efectul acestei instrucțiuni este scrierea peste un cuvânt aleator în memorie.

```
int  *p;
int i;

p = &i;
*p = 20;
```

➢ ‘p’ indică spre un întreg undeva în memorie (la adresa &i)

ptr2.c

ptr3.c

Operaţii cu pointeri

- ✓ Aritmetice:
  - adunare/scădere a unui întreg la/dintr-un pointer; incrementare/decrementare cu 1.
  - Scădere a unui *pointer* din altul (se poate obţine dimensiunea entităţii indicate)
- ✓ Relaţii (<, >, ==, etc.):
  - Comparaţii între doi *pointeri*: un pointer p1 este mai mare decât un *pointer* p2, dacă adresa pe care o conţine p1 este mai îndepărtată de începutul memoriei decât adresa conţinută de p2.
  - Compararea unui *pointer* cu o valoare constantă (spre exemplu, cu NULL pentru a verifica dacă *pointer*ul a fost iniţializat).

Utilitate

- ✓ *Pointer*ii sunt asociaţi cu lucrul cu memoria. Programatorul este cel care alocă şi eliberează memoria.
- ✓ În cazul în care scriem memoria fără a o aloca pot să apară probleme cu suprascrierea acesteia.

Alocarea memoriei

- **Alocarea automată**: entităţile temporare au alocată automat zona de memorie care este automat eliberată după ce acestea nu se mai folosesc (se părăseşte funcţia care le foloseşte).
- **Alocare statică**: variabilele alocate static nu sunt reiniţializate la fiecare apel al funcţiei în cadrul cărora au fost declarate.
- **Alocarea dinamică** a memoriei are următorul efect: blocuri de memorie (de orice dimensiune) pot fi alocate într-o zonă de memorie numită *heap* prin intermediul funcţiilor ***malloc(), realloc() şi free().*** Aceste blocuri de memorie pot fi reutilizate după ce zona de memorie a fost eliberată prin apelul funcţiei *free().*

"memory allocate"

Alocarea memoriei

- Sintaxa

```
#include <stdio.h>
#include <alloc.h>
void *malloc(size_t size);
```

- *size_t* tip utilizat pentru dimensiunea unor obiecte din memorie sau pentru contoare.
- *malloc()* alocă un bloc de *size* octeţi din zona de memorie *heap.*

- Permite unui program să aloce memorie explicit, atunci când este necesar şi **exact** la dimensiunea necesară (folosit pentru liste, arbori, etc.).
- În caz de succes *malloc()* **returnează un pointer la blocul de memorie nou alocat.**
- În caz de eroare (dacă nu există suficient spaţiu pentru noul bloc) *malloc()* returnează NULL. Conţinutul blocului rămâne neschimbat.
- Dacă dimensiunea argumentului este 0, *malloc()* returnează NULL.

Implementarea unui tablou bidimensional

```
int **array = (int **)malloc(rows * sizeof(int *));
for(i = 0; i < rows; ++i)
        array[i] = (int *)malloc(columns * sizeof(int));
```

```
int **array = (int **)malloc(rows * sizeof(int *));
array[0] = (int *)malloc(rows * columns * sizeof(int));
for(i = 1; i < rows; ++i)
        array[i] = array[0] + i * columns;
```

```
int *array = (int *)malloc(rows * columns * sizeof(int));
```

```
int (*array)[NO_OF_COLUMNS] =
        (int (*)[NO_OF_COLUMNS])malloc(rows * sizeof(*array));
```

Pointeri şi tablouri

```
int tab[30];
...
tab[25]=12;
...
printf("%d",tab[25]);
...
```

- ✓ În C un tablou este asociat cu un *pointer* care indică spre o adresă la care se găsesc elementele tabloului unele după altele.

tab[25]=12; ⇔ *(tab+25)=12;

- ✓ Compilatorul C nu face o diferenţiere evidentă între *pointer*i şi tablouri, ambele entităţi arată ca şi *pointer*ii.

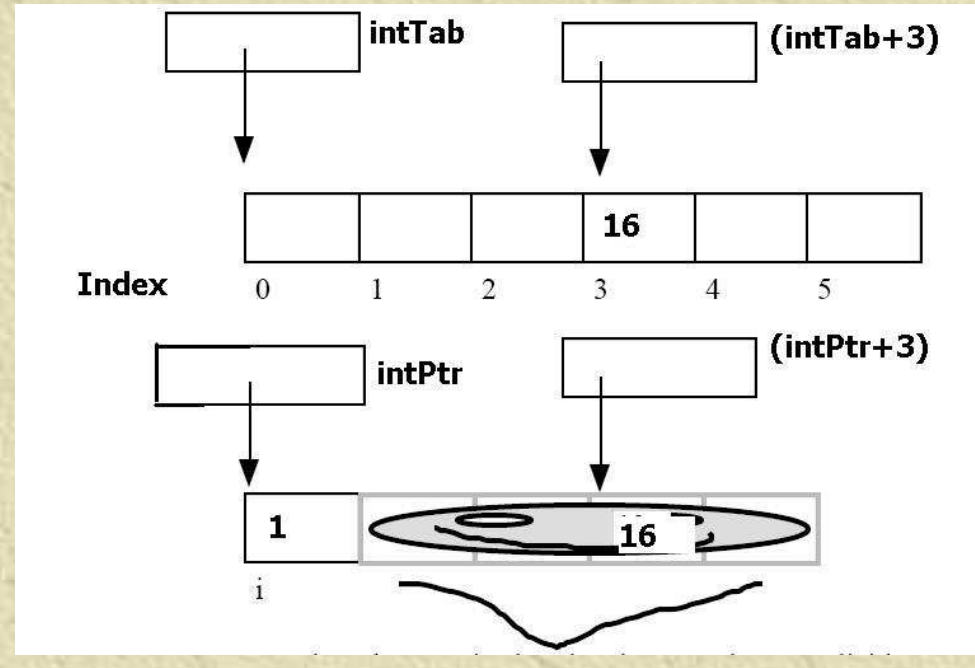
Pointeri şi tablouri

- ✓ **int intTab[6];**
- ✓ **int *intPtr;**
- ✓ **int i;**

- ✓ ‘intTab’ este un pointer către primul element al unui tabel de întregi, (int*).
- ✓ Valoarea catre care indică ‘intPtr’ este de tip (int*) şi intPtr este prevăzut să arate către un singur întreg ‘i’.
- ✓ intTab ? intPtr
- ✓ Din punctul de vedere al compilatorului nu este nici o diferenţă, ambele sunt *pointeri* (int*) şi este indiferent dacă e folosit [] sau + în asociere cu acestea.

Pointeri şi tablouri

- ✓ int intTab[6];
- ✓ int *intPtr;
- ✓ int i;
- ✓ intPtr = &i;
- ✓ intTab[3] = 16; ok
- ✓ intPtr[0] = 1; !? dar ok
- ✓ intPtr[3]= 16; greşit!
- ✓ ...



Biţii din memorie conţin probabil deja o anumită informaţie, iar 16 va fi scris peste informaţia existentă

Pointeri şi tablouri

- ✓ O distincţie fină între tablouri şi *pointeri*: *pointer*ul care reprezintă adresa de bază a unui tabel nu poate fi modificat în codul sursă (este un *pointer* 'const'). Constrângerea se aplică numelui tabelului.

```
int intTab[100];
int *p;
int i;

intTab = NULL;          nu
intTab = &i;            nu
intTab= intTab+1;       nu
intTab++;               nu

p = intTab;     da

p++;            da
p = NULL;       da
p = &i;         da
```

Pointeri şi şiruri

(1) char sir[6]; pentru un şir cu lungimea maximă 5 caractere;

(2) sir="continut sir"; /*greşit*/

printf("%s",sir);

(2) nu are ca efect plasarea textului 'continut sir' în spaţiul alocat prin declaraţia din (1).

- ✓ Se pierd cei 6 octeţi şi va seta 'sir' să indice către locul in care a fost plasat 'c' din 'continut sir'. Aparent nu e eroare, dar acest lucru se manifestă ulterior, atunci când este necesară modificarea textului sau accesarea şirului ca şi când s-ar afla în locaţia originală (caz în care s-ar obţine conţinutul aflat în spaţiul original neutilizat).
- ✓ Un şir în C este o parte de memorie care conţine caracterele memorate ca octeţi şi se termină cu un octet de valoare zero. O variabilă şir de caractere este un pointer care arată către primul caracter din şir.

Pointeri şi şiruri

- ✓ Un şir în C nu poate conţine caracterul ASCII zero pentru că astfel ar fi considerat un marcator de sfârşit de şir prematur; pentru a depăşi acest inconvenient se adaugă o variabilă care conţine lungimea şirului.
- ✓ Nu pot fi concatenate şiruri sau nu se poate copia un şir în altul decât folosind funcţii din bibliotecă sau funcţii care prelucrează şirul caracter cu caracter; semnul egal indică faptul că doi pointeri arată către aceeaşi zonă de memorie.
- ✓ Dacă nu se ţine cont de lunigmea necesară şirului, cu un octet în plus pentru marcarea sfârşitului de şir pot să apară erori şi programul să intre în zone de memorie necontrolate.

Pointeri şi funcţii

- int func(int a, float b);
- int *func(int a, float b); reprezintă o funcţie care returnează un pointer către un întreg
- int (*func)(int a, float b); reprezintă un pointer către o funcţie care returnează un întreg

Pointeri, tablouri, funcţii

✓ Parametrii tablou sunt pasaţi ca *pointeri*.

```
void func (int ParamTab[])   {
   ParamTab = NULL;     corect, dar nepotrivit
}


void func (int *ParamTab)   {
   ParamTab = NULL;     corect
}
```

Pointeri şi structuri

```
struct nume {
    char prenume [16] ;
    char init ;
    char num[20] ;
} ;
struct student {
    struct nume nume_st;
    int an;
    double debit;
} ;
struct student tabel[7000] ;
```

tabel[9] – al 10-a înregistrare din tabel – datele unui student (cu structura ‘student’);
tabel[9].nume_st – numele celui de-al 10-lea student din tabel (cu structura ‘nume’)
tabel[9].nume_st.init – iniţiala numelui celui de-al 10-lea student (un caracter)

```
typedef struct nume Nume ;
typedef struct student Student ;
typedef struct student *StudentPtr ;

Student tabel[7000];
```

Declarare

struct student *p_stu ; ***sau***

StudentPtr p_stu ;

Atribuire:

p_stu = &tabel[9] ;

Acces

p_stu->debit = p_stu * 1.02 ;

Pointeri, structuri, funcţii

- ✓ **O variabilă de tip structură se referă la întreaga structură (nu la adresă cum este cazul tablourilor).**
- ✓ **Atunci când o structură este pasată unei funcţii se pasează de fapt o copie a acesteia, iar modificările nu sunt reflectate în structura-original.**
- ✓ **Pentru a modifica o structură argumentul funcţiei trebuie să fie un *pointer.***

```
void clear_debit( struct student *p_student
) {
    p->debit = 0.0 ;
    return ;
}
    . . .
clear_debit( &tabel[9] ) ; …
```

Funcția **clear_debit** aşteaptă un *pointer* la **struct student** şi îl foloseşte pentru a pune pe zero componenta **stu_debit**.
În apelul funcției **clear_debit**, se utilizează ***operatorul de adresare &*** pentru a crea un *pointer* către al 10-lea student din 'tabel'.

Liste înlănţuite

Programarea calculatoarelor
Lăcrămioara Stoicu-Tivadar, 2007

Ce sunt listele înlănţuite? Care este structura acestora?

- ✓ Structuri de date fundamentale utilizate în programare
- ✓ O secvenţă de noduri (date şi adrese)

Avantajele utilizării listelor înlănţuite

- ✓ Posibilitatea de a insera şi de a şterge oricare dintre nodurile din listă la momentul dorit

Dezavantajul utilizării listelor înlănţuite

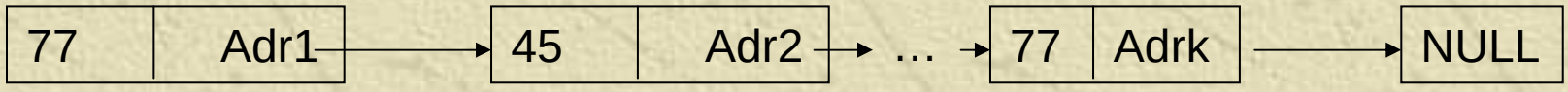
- ✓ Nu permit accesul direct la orice nod decât în manieră secvenţială
- ✓ nu se recomandă folosirea listelor înlănţuite pentru implementarea algoritmilor în care este important accesul rapid la valoarea elementelor (spre exemplu căutarea de tip *quicksort*).

Tipuri de liste înlănţuite

- ✓ liste liniare simplu înlănţuite
- ✓ liste liniare dublu înlănţuite
- ✓ liste circulare simplu înlănţuite
- ✓ liste circulare dublu înlănţuite.

Lista liniară simplu înlănţuită

- ✓în fiecare nod există un singur câmp referinţă care conţine adresa nodului următor, excepţie făcând ultimul nod din listă al cărui câmp referinţă conţine NULL sau 0.



Lista liniară simplu înlănţuită

- ✓ **ptr4.c**

- ✓ **Program care creează, concatenează, afişează, inversează o listă simplu înlănţuită.**

Alocarea memoriei

- ✓ Înlocuim implementarea prin tabel pentru baza de date cu studenţi cu o listă simplu înlănţuită: definim o structură nouă StuNod care conţine o structură 'student' şi un *pointer* către următorul StuNod din listă.

```
typedef struct stu_nod *StuNodPtr ;

typedef struct stu_nod {
    Student    sn_student ;
    StuNodPtr sn_next ;
} StuNod ;
```

Definim un pointer global, **tab_head** pentru a indica spre primul student din listă. Cum lista este iniţial goală, pointerul va fi pus pe zero (sau **NULL** –constantă simbolică din fişierul ***stddef.h***):

```
#include <stddef.h>
    . . .
StuNodPtr tab_head = NULL ;
```

Alocarea memoriei

- ✓ Este necesară o funcţie pentru a adăuga un student nou în listă (argumentul va fi noul student).
- ✓ Funcţia **malloc** asigură alocarea memoriei şi este declarată în fişierul ***stdlib.h***.

```
#include <stdlib.h>
   . . .
void add_student( Student stu_nou ) {
   StuNodPtr nod_nou ;
   nod_nou =
   (StuNodPtr) malloc( sizeof(StuNod) ) ;
   nod_nou->sn_student = stu_nou ;
   nod_nou->sn_next = tab_head ;
   tab_head = nod_nou;
}
```

Prima atribuire apelează funcția sistemului *malloc()* cu dimensiunea (în octeți) a unei structuri 'StuNode'. Valoarea returnată este un pointer spre noua zonă de memorie alocată, cel puțin la dimensiunea cerută.

În mod nefericit, pointerul este de tip 'char *', și nu 'StuNodePtr'. Pentru a remedia acest neajuns vom realiza o conversie (cast) a pointerului din tipul real în timpul dorit prin: 'StuNodePtr' în fața lui *malloc()*.

Acțiunile din cod:

- Copiază argumentul structură student în componenta structură student a nodului
- Leagă nodurile cu entitatea din capul listei
- Setează pointerul 'head' către noul nod creat.

Alocarea memoriei

- ✓ În final, este nevoie de un mod de eliminare a unui nod al structurii pentru cazul în care un student este şters din listă
- ✓ Exemplul se limitează doar la a şterge spaţiul asociat nodului, fără a intra în detalii legate de găsirea nodului în listă şi de "dezlegare" corectă a sa .

```
void free_node( StuNodePtr p_stu ) {
    (void) free( (char *) p_stu ) ;
}
```

Funcţia *free* se găseşte în biblioteca standard şi are ca efect eliberarea spaţiului asociat cu pointerul *p_stu*. Pointerul este convertit către tipul (char *), care este tipul de pointer aşteptat de *free*. Valoarea returnată de *free* este un întreg, dar valoarea este foarte rar (poate niciodată) utilizată. Pentru a indica faptul că nu suntem interesaţi de valoare precizăm (prin conversie, cast) tipul valorii returnate ca *void*.
Odată eliberată o zonă de memorie nu mai este permisă nici o referire la aceasta, rezultatul contrar fiind un comportament nedefinit al programului.

Pointeri şi optimizare

- Utilizarea *pointer*ilor pentru a transmite modificarea variabilelor poate avea ca efect
  - economia de memorie şi
  - un timp de execuţie mai scurt.
- În cazul în care e vorba de un întreg nu se simte acest lucru pentru că un întreg are aceeaşi lungime ca şi *pointer*ul care este copiat. Avantajul este pus în evidenţă în cazul unor variabile mai complicate, în programe care trebuie să fie executate rapid. Spre exemplu, în cazul utilizării structurilor.

Pointeri şi optimizare

- struct tip1 {int Part1,char* Part2}; defineşte un tip care este format dintr-un întreg şi un şir
- struct tip1 var3; creează o variabilă de tipul anterior definit
- var3.Part2="sirutil";
  printf("%s", var3.Part2); prezintă modul de acces la componentele variabilei structură
- Comp3->Comp4 ⇔ (*Comp3).Comp4 - din structura către care indic Comp3 se foloseşte componenta Comp4.

Pointeri şi optimizare

- ✓ Dacă spre ‘var3’ indică un pointer

struct tip1 *Pointer2;

Pointer2=&var3;

Atunci se poate folosi exprimarea

Pointer2->Part1=96;

pentru a seta componenta ‘Part1’ din ‘var3’ la 96 în loc de

(*Pointer2).Part1=96;

În cazul de faţă diferenţa nu este foarte mare, dar când sunt structuri care conţin ele însele pointeri atunci nu mai sunt folosite multe paranteze unele în altele.

Pointer3->Part11->Part12-Part13 în loc de (*(*(*Pointer3).Part11).Part12).Part13

Pointerii şi calificatorul *const*

- ✓ În asociere cu pointerii calificatorii, în speţă *const* - pot fi aplicaţi în două moduri:
  - fie pentru entitatea către care indică pointerul (pointer către const)
  - fie către pointer însuşi (pointer constant)

Pointerii şi calificatorul *const*

int i;

const int ci = 1; ci este o constantă întreagă

int *pi; pi este un pointer către un întreg

const int *pci; pci este un pointer către o constantă întreagă

int *const cpi = &i; cpi este un pointer constant către un întreg

const int *const cpci = &ci; cpci este un pointer constant către un constant întreg

Aplicaţii

- ✓ ptr5.c – ptr7.c

Sumar C13

- ✓ **Fişiere**